

Chapter 5

Instruction Set (part 2)

Instruction Set – Basic Categories

- Non system-level instructions in a processor can be typically classified into three basic groups:

Data Transfer

ARM examples:

```
MOV R1, R0
STR R0, [R2, #4]
LDR R1, [R2]
```

Data Processing

ARM examples:

```
ADD R0, R1, R2
SUB R1, R2, #3
EOR R3, R3, R2
```

Program Control

ARM examples:

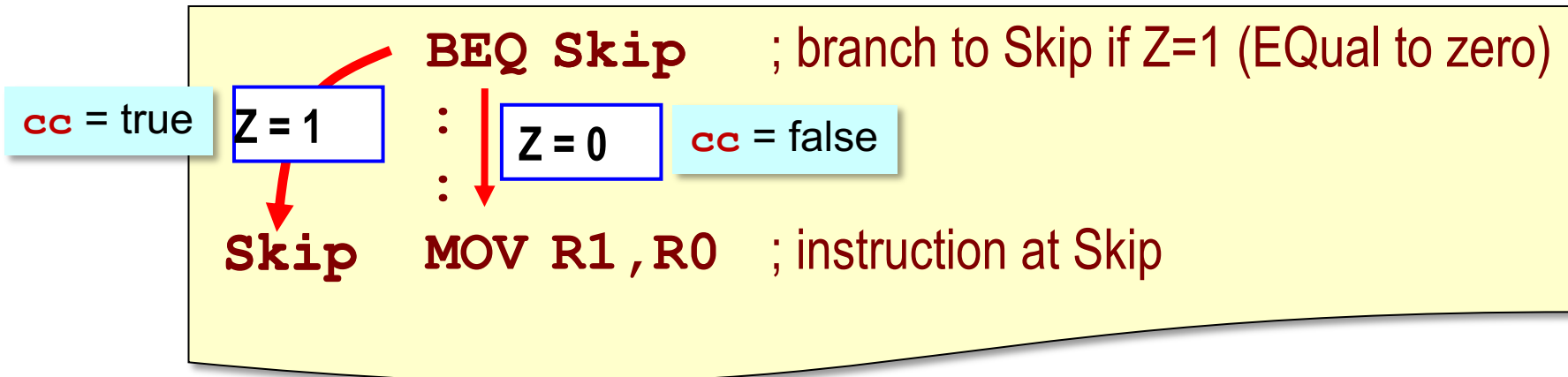
```
B Back
BNE Loop
BL Routine
```

**Covered in
Modular
Programming**

- Data transfer – instructions that move data between registers.
- Data processing – instructions that modify the data using arithmetic, logical or shift operations.
- Program control** – instructions that alter the normal sequential execution flow of a program.

Conditional Branch (Bcc)

- ARM provides conditional branch using **Bcc**.
- If the condition specified in the condition field (**cc**) is **true**, a displacement is added to the **PC**, otherwise next instruction is executed.
- Bcc** uses **PC-relative** addressing mode with a displacement range of $\pm 32\text{MB}$.
- The **PC** value used to compute required displacement is **8 bytes** ahead of the current **Bcc** being executed.
- Bcc** is used with **address labels** that allows the assembler to compute the required displacement values.



Different Bcc Conditions

- There are 15 possible conditional tests for **Bcc**.

Suffix	Flags	Meaning	
EQ	Z = 1	Equal	
NE	Z = 0	Not equal	
CS or HS	C = 1	Higher or same, unsigned	Unsigned comparison
CC or LO	C = 0	Lower, unsigned	
MI	N = 1	Negative	
PL	N = 0	Positive or zero	
VS	V = 1	Overflow	
VC	V = 0	No overflow	
HI	C = 1 and Z = 0	Higher, unsigned	Unsigned comparison
LS	C = 0 or Z = 1	Lower or same, unsigned	
GE	N = V	Greater than or equal, signed	Signed comparison
LT	N != V	Less than, signed	
GT	Z = 0 and N = V	Greater than, signed	
LE	Z = 1 and N != V	Less than or equal, signed	
AL	Can have any value	Always. This is the default when no suffix is specified.	

CMP

- **CMP subtracts** the source operand from the destination register and sets the **CC** flags according to the results.
- Destination register remain **unmodified** after **CMP**.
- CC flags affected in the same manner as the **subtract** instruction (**SUBS**).

	SUBS	R1 , R1 , R2
	BGE	R1 ≥ R2
	:	
R1 ≥ R2	:	

**R1 modified to achieve
desired flow control**

	CMP	R1 , R2
	BGE	R1 ≥ R2
	:	
R1 ≥ R2	:	

**Same flow control
but R1 unchanged**

Analyzing Code Constructs

Given the code segment shown, how many times the instruction ADDS is executed

A) 0

Answer is C

```
MOV    R0 , #0xFFFFFFFF
Loop   ADDS R0 , R0 , #1
       BNE  Loop
```

B) 1

C) 2

D) 3

Suffix	Flags	Meaning	
EQ	Z = 1	Equal	
NE	Z = 0	Not equal	
CS or HS	C = 1	Higher or same, unsigned	Unsigned comparison
CC or LO	C = 0	Lower, unsigned	
MI	N = 1	Negative	
PL	N = 0	Positive or zero	
VS	V = 1	Overflow	
VC	V = 0	No overflow	
HI	C = 1 and Z = 0	Higher, unsigned	Unsigned comparison
LS	C = 0 or Z = 1	Lower or same, unsigned	
GE	N = V	Greater than or equal, signed	Signed comparison
LT	N != V	Less than, signed	
GT	Z = 0 and N = V	Greater than, signed	
LE	Z = 1 and N != V	Less than or equal, signed	
AL	Can have any value	Always. This is the default when no suffix is specified.	

CMP instruction

Given the code segment shown, how many times the instruction CMP is executed

A) 0

B) 1

Answer is C

C) 2

D) 3

```
      SUB    R1 , R1 , R1
Loop  CMP    R1 , #1
      BEQ    Exit
      ADD    R1 , R1 , #1
      B      Loop
Exit  ...
```

Conditional Test (Bcc)

What is the content of R1 that will avoid the execution of MOV R2,R3

A) R1=0XFFFFFFFFFD

B) R1=0XFFFFFFFFFE

C) R1=0XFFFFFFFFF

D) R1=0x00000000

```
CMP    R1 , #0xFFFFFFFFFE
BLT     NEXT
MOV     R2 , R3
NEXT    . . .
```

Answer is A

Conditional Test (Bcc)

What is the content of R2 that will avoid the execution of MOV R2,R3

A) R2=0X00000001

B) R2=0XFFFFFFFFFE

C) R2=0xFFFFFFFFFF

D) R2=0x00000000

```
MOV    R1 , #0
CMP    R1 , R2
BHS    NEXT
MOV    R2 , R3
NEXT    . . .
```

Answer is D

Write an ARM assembly language program to compute the average value of eight numbers in an array.

(1) Your program should be written based on the following specifications:

- a) These numbers are word-sized unsigned integers (i.e. 32-bit unsigned numbers).
- b) The eight numbers are in consecutive memory locations starting at address 0x0100.
- c) The average (quotient) and remainder are stored in registers R0 and R1 respectively.

0x00000001
0x00000002
0x00000003
0x00000004
0x00000005
0x00000006
0x00000007
0x00000008

Write an ARM assembly language program to compute the average value of eight numbers in an array.

(1) Your program should be written based on the following specifications:

- a) These numbers are word-sized unsigned integers (i.e. 32-bit unsigned numbers).
- b) The eight numbers are in consecutive memory locations starting at address 0x0100.
- c) The average (quotient) and remainder are stored in registers R0 and R1 respectively.

0x00000001
0x00000002
0x00000003
0x00000004
0x00000005
0x00000006
0x00000007
0x00000008

Q2.4

Write an ARM assembly language program to compute the average value of eight numbers in an array.

- (1) Your program should be written based on the following specifications:
 - a) These numbers are word-sized unsigned integers (i.e. 32-bit unsigned numbers).
 - b) The eight numbers are in consecutive memory locations starting at address 0x0100.
 - c) The average (quotient) and remainder are stored in registers R0 and R1 respectively.

```
Start      MOV R1,#0x100    ; initialize array pointer to start of array
           MOV R0,#0        ; initialize cumulating register R0 to zero
           MOV R2,#8        ; initialize loop counter for 8 loops
SumLoop    LDR R3,[R1],#4    ; get current element and increment array pointer
           ADD R0,R0,R3      ; add array element to current sum in R0
           SUBS R2,R2,#1     ; decrement loop counter R2 (check Z flag)
           BNE SumLoop      ; branch back until count is 0
GetQnR     MOV R1,R0        ; make copy of cumulated sum in R1
           AND R1,R1,#0x07  ; clear all bits except lowest 3 bits to get
                           ; remainder in R1
           MOV R0,R0,LSR #3 ; logical shift right by 3 bits to divide by 8
                           ; to get quotient
```

0x00000001
0x00000002
0x00000003
0x00000004
0x00000005
0x00000006
0x00000007
0x00000008